Big data processing and analytics

Antonio Villella

Anno accademico 2026–2027

Indice

1 Introduzione ai Big Data **6**
- 1.1 Che cosa sono i Big Data? . . . 6
- 1.2 Fonti e Generatori di Big Data . . . 6
- 1.3 Le "V" dei Big Data . . . 7
- 1.4 Framework Big Data: Motivazioni e Sfide . . . 7
  - 1.4.1 Perché abbiamo bisogno di nuovi framework? . . . 7
  - 1.4.2 Il principio della Data Locality . . . 7
  - 1.4.3 Architettura dei Data Center e Scalabilità . . . 8
  - 1.4.4 Le sfide tecniche . . . 8
  - 1.4.5 Il problema tipico nei Big Data . . . 8

2 Big Data Architectures **9**
- 2.1 Introduzione alle Architetture Big Data . . . 9
  - 2.1.1 Quando utilizzare un'architettura Big Data . . . 9
- 2.2 La Lambda Architecture . . . 9
  - 2.2.1 Definizioni a confronto . . . 10
  - 2.2.2 Il Modello dei Dati nella Lambda Architecture . . . 10
  - 2.2.3 Requisiti dell'Architettura . . . 11

3 Hadoop e MapReduce **12**
- 3.1 Introduzione al Framework Hadoop . . . 12
- 3.2 Il Principio della Data Locality . . . 13
- 3.3 Campi di Applicazione di MapReduce . . . 13
- 3.4 Il Paradigma di Programmazione MapReduce . . . 14
  - 3.4.1 Struttura Dati: Coppie Chiave-Valore . . . 14
  - 3.4.2 Le Fasi di Elaborazione . . . 14
- 3.5 Esempio Pratico: Il Problema del Word Count . . . 14

4 Sviluppo di Applicazioni MapReduce in Hadoop **16**
- 4.1 Il Ruolo del Framework e dello Sviluppatore . . . 16
- 4.2 Architettura di un Programma MapReduce . . . 16
  - 4.2.1 Definizione delle Classi . . . 16
  - 4.2.2 Istanze e Terminologia . . . 17
  - 4.2.3 Il Driver come Entry Point . . . 17
- 4.3 Gestione dell'Output Intermedio e Finale . . . 17
  - 4.3.1 Dati Intermedi (Transient Data) . . . 17
  - 4.3.2 Dati Finali . . . 17
- 4.4 Lettura dei Dati: La classe InputFormat . . . 18
- 4.5 Parallelismo ed Esecuzione: Task e Input Split . . . 18
  - 4.5.1 Grado di Parallelismo della fase di Map . . . 18

4.6 Tipi di Dato in MapReduce 19
4.6.1 Tipi di Dato Personalizzati (Personalized Data Types) 19
4.6.2 Tipi di Dato Personalizzati (Personalized Data Types) 19
4.7 Implementazione del Reducer 19
4.7.1 Esempio: Word Count Reducer 20
4.8 L'Uso esclusivo del Map: Map-Only Job 20
4.9 Ottimizzazione del Traffico di Rete: Il Combiner 21
4.9.1 Implementazione del Combiner 21
4.10 Ottimizzazione Avanzata: L'In-Mapper Combiner 22
4.10.1 Workflow dell'In-Mapper Combiner 22
4.10.2 Considerazioni Critiche sulle Prestazioni e Memoria Principale 22
4.11 Gestione del Contesto e Statistiche 22
4.11.1 Condivisione di Parametri globali 22
4.11.2 Contatori (Counters) 23

Introduzione al Corso e Modalità d'Esame

Informazioni Generali sul Corso

Il corso affronta le sfide e le opportunità legate all'analisi e alla gestione di enormi quantità di dati (Big Data). Vengono introdotti non solo modelli di dati per l'archiviazione (database relazionali e non relazionali scalabili), ma soprattutto i nuovi paradigmi di programmazione e i framework distribuiti necessari per l'elaborazione dei dati, come Hadoop e Spark.

Docenti e Materiale Didattico

I docenti del corso sono Paolo Garza, Hadi Ibrahim e Lorenzo Innocenti. Per eventuali dubbi, i docenti sono a disposizione durante le pause o al termine della lezione, oppure tramite appuntamento via email.

Tutto il materiale di riferimento (slide, esercizi, attività di laboratorio ed esami passati) è disponibile sulla pagina web dedicata del corso su `minds.polito.it`. Le videolezioni sono rese disponibili sul Portale della Didattica. Sebbene vi siano dei libri di testo suggeriti (come *Spark: The Definitive Guide* o *Hadoop, The Definitive Guide*), le slide e gli esercizi forniti sono considerati sufficienti per l'apprendimento e il superamento dell'esame.

Osservazione 0.1 (Metodo di studio). L'impegno stimato per il corso prevede che per ogni credito formativo (CFU) si dedichino circa 10 ore in aula/laboratorio e 15 ore di studio personale (in totale 25 ore per CFU). Il professore consiglia vivamente di provare a risolvere gli esercizi e le attività di laboratorio in autonomia *prima* di consultare le soluzioni, al fine di assimilare i "pattern" fondamentali di risoluzione, necessari per costruire applicazioni complesse.

Prerequisiti e Competenze Pregresse

Per affrontare il corso con successo sono richieste alcune competenze di base in ambito di programmazione e basi di dati:

- **Java:** Necessario per scrivere applicazioni MapReduce su Hadoop. È richiesta una conoscenza di base (es. cicli e operazioni elementari sulle stringhe).
- **Python:** Utilizzato per scrivere applicazioni su Spark. Anche in questo caso sono richieste operazioni di base (filtraggio, trasformazione dei dati e calcolo di statistiche).
- **Database e SQL:** Utile per comprendere il modello relazionale e, in particolar modo, per interfacciarsi con le API di Spark SQL.

Programma e Argomenti del Corso

Il corso è strutturato per insegnare a scrivere applicazioni software (non solo l'architettura dei sistemi) ed è diviso in diversi macro-argomenti:

Hadoop e MapReduce

Verrà studiata l'architettura base di Hadoop e il paradigma di programmazione MapReduce (in Java).

Osservazione 0.2 (Perché studiare ancora MapReduce?). Sebbene oggi non sia il framework più performante, il professore sottolinea che esistono ancora moltissimi sistemi legacy basati su questo paradigma (in alcuni casi con 50 anni di dati storici archiviati, la cui migrazione richiederebbe settimane o mesi, un po' come accade per i vecchi software in COBOL). È quindi un'abilità fondamentale per un ingegnere dei dati saper mettere mano, leggere e aggiornare questo tipo di codice.

Apache Spark

Spark è attualmente uno dei framework più utilizzati per l'analisi di enormi quantità di dati. Il corso esplorerà la sua architettura e insegnerà a scrivere applicazioni utilizzando due diversi approcci:

- **RDD (Resilient Distributed Datasets):** La versione precedente delle API di Spark (scritta in Python/Java). Secondo il docente, gli RDD sono fondamentali da studiare non solo per via del codice legacy, ma anche perché permettono di comprendere esattamente quanti e quali dati viaggiano sulla rete, fattore chiave per valutare l'efficienza di un'applicazione Big Data.
- **Spark SQL e DataFrames:** Le API più recenti. Permettono di utilizzare direttamente il linguaggio SQL per processare i dati. Spesso, conoscendo già l'SQL, questo approccio risulta molto più semplice ed è il più utilizzato oggi in produzione.

Librerie Avanzate: MLlib e Spark Streaming

- **MLlib (Machine Learning):** La libreria di machine learning scalabile di Apache Spark. Il professore precisa che l'obiettivo non è imparare a scrivere nuovi algoritmi di machine learning distribuiti da zero, ma imparare a usare quelli già presenti e parallelizzabili per analizzare grandi quantità di dati.
- **Spark Streaming:** Strumenti per l'analisi di flussi continui di dati. La transizione concettuale è semplice: una volta compreso come analizzare i dati "in batch" con Spark, l'approccio allo streaming risulta identico; cambia unicamente la sorgente di input (che passa dall'essere un grande file statico a un flusso continuo).

Regole e Modalità d'Esame

L'esame consiste unicamente in una prova scritta erogata tramite piattaforma d'ateneo (Exam Platform con browser protetto *Lockdown*), della durata di **1,5 ore (90 minuti)**. Per sostenere l'esame è necessario portare il proprio PC.

Il punteggio massimo è di **31 punti**, così strutturato:

- **2 Esercizi di Programmazione (max 27 punti):** Basati sulla progettazione di programmi Hadoop/MapReduce e/o API Spark. Spesso un singolo programma richiederà di risolvere due sotto-problemi, ed è valutata l'abilità dello studente nel riutilizzare il codice in modo efficiente per entrambi.
- **2 Domande Teoriche a risposta multipla (max 4 punti):** Vertono su tutti gli argomenti (architetture, RDD, HDFS, ecc.). Spesso queste domande presentano porzioni di codice di cui viene richiesto di prevedere l'output.

Osservazione 0.3 (Esame a Libro Aperto). L'esame è "open book", ovvero è consentito consultare libri, appunti e materiale cartaceo stampato. **Tuttavia, non è permesso alcun dispositivo elettronico** (cellulari, tablet o altri PC) al di fuori di quello utilizzato per l'esame. Il docente avvisa severamente che qualsiasi tentativo di copiatura ("cheating") sarà sanzionato con l'annullamento dell'esame e la potenziale sospensione di un anno dagli appelli (con conseguente perdita di eventuali borse di studio).

Capitolo 1

Introduzione ai Big Data

1.1 Che cosa sono i Big Data?

Il termine **Big Data** è generalmente associato a enormi quantità di dati ("petabytes") raccolte in grandi collezioni. Tuttavia, il concetto non si limita alla sola quantità, ma coinvolge diverse dimensioni fondamentali:

- **Storage**: i dati devono essere memorizzati utilizzando approcci efficienti ed efficaci. Spesso non si utilizzano i tradizionali database relazionali, ma ci si affida a *distributed file system* o database NoSQL.
- **Analytics**: l'obiettivo finale è estrarre conoscenza e valore dai dati attraverso metodologie di analisi.
- **Technologies**: sono necessarie nuove tecnologie e framework per elaborare e analizzare questi dati in modo scalabile.

Inizialmente, i framework per i Big Data sono stati sviluppati dalle grandi aziende di internet (le "Big Tech" come Google e Meta). Oggi, tuttavia, la necessità di analizzare enormi moli di dati riguarda qualsiasi settore, dalle banche agli impianti industriali.

Osservazione 1.1. **Sfruttare i dati esistenti per nuovi task**
Il professore sottolinea che un buon *data scientist* o *data engineer* deve saper utilizzare i dati già a disposizione per risolvere nuovi problemi. Ad esempio, Google ha iniziato a raccogliere le query degli utenti per fornire risposte migliori, ma ha poi utilizzato gli stessi dati per fare *nowcasting* (previsione a brevissimo termine), stimando in tempo reale quante persone fossero affette da influenza in una determinata area.

1.2 Fonti e Generatori di Big Data

Esistono diverse fonti in grado di generare enormi quantità di dati, che tipicamente vengono poi convogliate e memorizzate in un singolo Data Center. Le principali categorie includono:

1. **User Generated Content (Web & Mobile)**: contenuti generati dagli utenti su piattaforme come Facebook, Instagram, Yelp, TripAdvisor, Twitter e YouTube.
2. **Health and scientific computing**: dati ad alta risoluzione provenienti dal settore medico, immagini satellitari (utili ad esempio per identificare aree allagate o bruciate), microscopi e telescopi.

3. **Log files**: file di testo generati da web server o sistemi automatici che tracciano eventi e accessi.
4. **Internet Of Things (IoT)**: reti di sensori (*Sensor networks*), RFID e *smart meters*. Anche se ogni singolo sensore invia una quantità minuscola di dati, l'aggregazione di miliardi di sensori genera moli di dati enormi.

1.3 Le "V" dei Big Data

Il contesto dei Big Data è tradizionalmente descritto attraverso le **3V**:

- **Volume (Scale of data)**: la scala dimensionale dei dati. L'unità di misura tipica entra nell'ordine dei Petabyte (PB). Intorno al 2012, Google processava oltre 100 PB di dati al giorno, mentre Facebook oltre 10 PB.
- **Variety (Different forms of data)**: l'eterogeneità dei dati in ingresso. Un sistema Big Data dovrebbe poter gestire testo, immagini, grafi e log.
- **Velocity (Analysis of streaming data)**: la velocità con cui i dati vengono generati e la necessità di analizzare flussi di dati continui (stream) elaborando milioni di eventi in pochi millisecondi.

A queste, nel tempo, sono state aggiunte altre due dimensioni:

- **Veracity (Uncertainty of data)**: la qualità dei dati.
- **Value**: la capacità di estrarre informazioni utili dal dato.

Osservazione 1.2. **Veracity e qualità del dato**
Il professore ricorda il principio *"Garbage in, garbage out"*: se l'input è rumoroso o di scarsa qualità, anche l'output degli algoritmi di Machine Learning sarà pessimo. Inoltre, i dati grezzi raramente sono pronti per l'analisi: in una tipica pipeline di Data Science, dal 60% al 70% del tempo è dedicato alla pulizia del dato (*data cleaning*) e alla sua trasformazione in vettori o matrici.

1.4 Framework Big Data: Motivazioni e Sfide

1.4.1 Perché abbiamo bisogno di nuovi framework?

Nel passato avevamo già a disposizione database relazionali efficienti, sistemi NoSQL e l'High-Performance Computing (HPC). Tuttavia, questi sistemi mostrano dei limiti con moli di dati eccezionali. Ad esempio, per indicizzare 10 miliardi di pagine web (da circa 20 KB l'una), un *full scan* sequenziale su un disco rigido tradizionale o l'estrazione a blocchi nella memoria RAM (come nel classico *Data Mining*) richiederebbe un tempo inaccettabile.

Occorreva passare da un'architettura a nodo singolo (*single-node architecture*) a un approccio fortemente distribuito.

1.4.2 Il principio della Data Locality

Il concetto base sviluppato da Google per i framework Big Data è la **Data Locality**.

Definizione 1.1 (Data Locality)**.** Invece di inviare enormi quantità di dati attraverso la rete verso i server che contengono il codice da eseguire, si invia il codice (che è di dimensioni molto ridotte) direttamente ai server in cui i dati sono fisicamente memorizzati.

Inviare troppi dati sulla rete crea dei colli di bottiglia (*network bottleneck*) che rendono l'applicazione lentissima.

1.4.3 Architettura dei Data Center e Scalabilità

I sistemi Big Data utilizzano **commodity hardware**, ovvero server standard, economici e facilmente sostituibili in caso di guasto. Questi server sono raggruppati in rack e interconnessi tramite switch di rete.

Osservazione 1.3. **High-Performance Computing (HPC) vs Big Data Framework** L'HPC è indicato per compiti intensivi dal punto di vista computazionale (es. calcolare miliardi di operazioni in virgola mobile al secondo su un set di dati relativamente piccolo) e si basa sulla *scalabilità verticale*. I framework Big Data (come Hadoop o Spark) sono invece adatti a carichi di lavoro **data-intensive** e si basano sulla **scalabilità orizzontale**, dove la metrica principale è quanti dati il sistema riesce ad analizzare in un certo lasso di tempo.

1.4.4 Le sfide tecniche

Secondo le slide, le principali sfide affrontate dai framework Big Data sono:

- Calcolo distribuito e parallelizzazione;
- Storage distribuito di dataset di dimensioni enormi (Terabyte, Petabyte);
- Gestione dei guasti dei nodi (*Node failure management*);
- Colli di bottiglia della rete (*Network bottleneck*);
- Formati di input eterogenei (*Data diversity & heterogeneity*).

Il vantaggio dell'utilizzo di framework come Spark è che lo sviluppatore non deve occuparsi direttamente dello scheduling dell'applicazione, della gestione della rete o dei fallimenti hardware, poiché vengono gestiti automaticamente dal framework.

1.4.5 Il problema tipico nei Big Data

Non tutti i problemi possono essere risolti efficientemente con un approccio Big Data (ad esempio, quando è necessaria una visione "globale" immediata dei dati per prendere una decisione). Inoltre, le funzioni applicate ai dati distribuiti devono soddisfare specifici vincoli matematici: devono essere **commutative** e **associative**.

Un problema tipico che ben si adatta a questi framework segue questo schema operativo:

1. Iterare su una quantità enorme di record/oggetti;
2. Estrarre l'informazione di interesse da ogni record (*Map*);
3. Aggregare i risultati intermedi (*Reduce*);
4. Generare l'output finale / risultato globale.

Capitolo 2

Big Data Architectures

2.1 Introduzione alle Architetture Big Data

Un'architettura Big Data è progettata per gestire l'ingestione, l'elaborazione e l'analisi di dati che risultano troppo grandi o complessi per i sistemi di database tradizionali.

Le soluzioni Big Data tipicamente coinvolgono una o più delle seguenti tipologie di carico di lavoro (workload):

- **Batch processing** di sorgenti Big Data *at rest* (dati a riposo/archiviati).
- **Real-time processing** di Big Data *in motion* (dati in movimento/streaming).
- **Interactive exploration** (esplorazione interattiva) dei Big Data.
- **Predictive analytics e machine learning**.

2.1.1 Quando utilizzare un'architettura Big Data

È opportuno considerare l'adozione di un'architettura Big Data quando si ha la necessità di:

- Memorizzare ed elaborare dati in **volumi troppo ampi** per un database tradizionale.
- Trasformare **dati non strutturati** per finalità di analisi e reportistica.
- Catturare, elaborare e analizzare **flussi illimitati di dati (unbounded streams)** in tempo reale o con bassa latenza (low latency).

Osservazione 2.1 (Nota del professore: Scegliere lo strumento giusto). Il professore sottolinea un aspetto fondamentale per un ingegnere: è cruciale utilizzare la soluzione corretta per il problema specifico che si deve risolvere. Non bisogna adottare un framework Big Data solo perché "lo fanno tutti" o perché è una tecnologia di tendenza, specialmente se i dati da gestire sono di piccole dimensioni. Utilizzare un'architettura complessa quando non è necessaria porta unicamente a uno spreco di tempo e di risorse computazionali.

2.2 La Lambda Architecture

L'architettura Big Data più frequentemente utilizzata (secondo il professore, presente in circa il 99% dei sistemi basati su framework Big Data) è la **Lambda Architecture**. È stata proposta per la prima volta da Nathan Marz nel 2011 e, seppur basata su idee molto semplici, risulta estremamente efficace ed efficiente.

2.2.1 Definizioni a confronto

Per comprendere appieno questa architettura, è utile analizzare tre diverse definizioni fornite in letteratura e nel settore industriale:

Definizione 2.1 (Definizione #1 - Nathan Marz)**.** I database NoSQL (come Cassandra) ottengono la loro scalabilità offrendo un **modello di dati molto più limitato** rispetto al classico SQL, rendendo complessa l'adattabilità delle applicazioni. Inoltre, essendo database mutabili, **non sono tolleranti agli errori umani (human-fault tolerant)**. La Lambda Architecture nasce dall'uso intelligente e congiunto di diversi strumenti (sistemi di calcolo su larga scala e database) per produrre sistemi scalabili per problemi di dati arbitrari, garantendo la tolleranza agli errori umani con il minimo della complessità.

Definizione 2.2 (Definizione #2 - Databricks)**.** La Lambda Architecture è un modo per elaborare quantità massive di dati fornendo accesso sia a metodi di **batch-processing** che di **stream-processing** attraverso un **approccio ibrido**. È utilizzata per risolvere il problema del calcolo di funzioni arbitrarie.

Definizione 2.3 (Definizione #3 - Wikipedia)**.** È un'architettura di elaborazione dati progettata per gestire quantità massive di dati sfruttando i vantaggi sia dei metodi batch che stream. Cerca di **bilanciare latenza, throughput e tolleranza ai guasti (fault-tolerance)** in questo modo:

- Il *batch processing* fornisce viste accurate e complete sui dati archiviati (batch data).
- Lo *stream processing* in tempo reale fornisce viste sui dati online.

Gli output di queste due viste vengono poi uniti (joined) prima della presentazione finale all'utente.

2.2.2 Il Modello dei Dati nella Lambda Architecture

La Lambda Architecture si basa fortemente su un modello di dati specifico: utilizza una **sorgente dati immutabile e append-only** che funge da sistema di registrazione (system of record).

È pensata per l'ingestione e l'elaborazione di **eventi dotati di timestamp**, i quali vengono accodati (appended) agli eventi esistenti piuttosto che sovrascriverli. Lo stato del sistema viene determinato dal naturale ordinamento temporale dei dati.

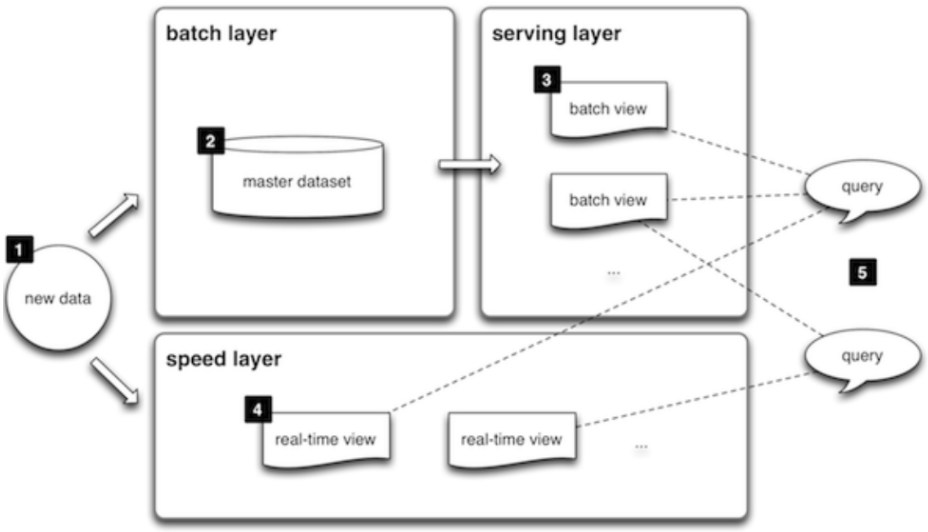


Figura 2.1: Struttura e livelli della Lambda Architecture.

2.2.3 Requisiti dell'Architettura

La Lambda Architecture è progettata per soddisfare i seguenti requisiti fondamentali:

- **Tolleranza ai guasti (Fault-tolerance):** Deve resistere sia ai guasti hardware (hardware failures) sia agli errori umani (human errors).
- **Supporto per molteplici casi d'uso:** Deve gestire sia interrogazioni a bassa latenza sia aggiornamenti complessi sui dati storici.
- **Capacità di Linear Scale-out:** Deve poter scalare orizzontalmente in modo lineare.

Osservazione 2.2 (Implementazione dello Scale-out). Come specificato dal professore, la scalabilità "scale-out" (ovvero l'aggiunta di nuovi nodi al cluster anziché il potenziamento del singolo nodo) viene implementata utilizzando framework e tecnologie progettate nativamente per questo scopo, come i database NoSQL e i framework Hadoop o Spark. Quando si processano dati nel livello batch, si effettua tipicamente un *full scan* (scansione completa) dell'intera collezione di dati in input, analizzando un record alla volta per ottenere il risultato finale.

Capitolo 3

Hadoop e MapReduce

3.1 Introduzione al Framework Hadoop

Il framework Hadoop, pur essendo stato proposto molti anni fa, introduce concetti di base che rimangono fondamentali nel mondo dell'elaborazione dei Big Data. I componenti principali di Hadoop includono un'infrastruttura di elaborazione distribuita, interamente basata sul paradigma di programmazione MapReduce, e il file system distribuito HDFS (Hadoop Distributed File System). Inoltre, all'ecosistema Hadoop sono associati molti altri progetti paralleli, come Spark e Hive (un database relazionale distribuito che sfrutta applicazioni MapReduce).

L'infrastruttura Hadoop è caratterizzata da alcune proprietà fondamentali:

- **Astrazione ad alto livello (Separazione del *What* dal *How*):** Hadoop permette ai programmatori di concentrarsi esclusivamente su *cosa* calcolare (implementando il paradigma MapReduce), mentre l'infrastruttura di Hadoop gestisce automaticamente il *come*, occupandosi di tutte le complessità dei sistemi distribuiti, come lo scheduling dei task e la sincronizzazione.
- **Tolleranza ai guasti (Fault-tolerance):** I fallimenti dei singoli server (nodi) o dei task vengono gestiti automaticamente dal sistema Hadoop. Ad esempio, per evitare "single points of failure", vengono mantenute delle copie di backup (shadow copies/repliche) su server diversi che subentrano immediatamente nel caso in cui un nodo principale smetta di funzionare.
- **Alta disponibilità (High availability):** Il file system HDFS garantisce che lo storage distribuito sia sempre altamente disponibile e resiliente agli errori.

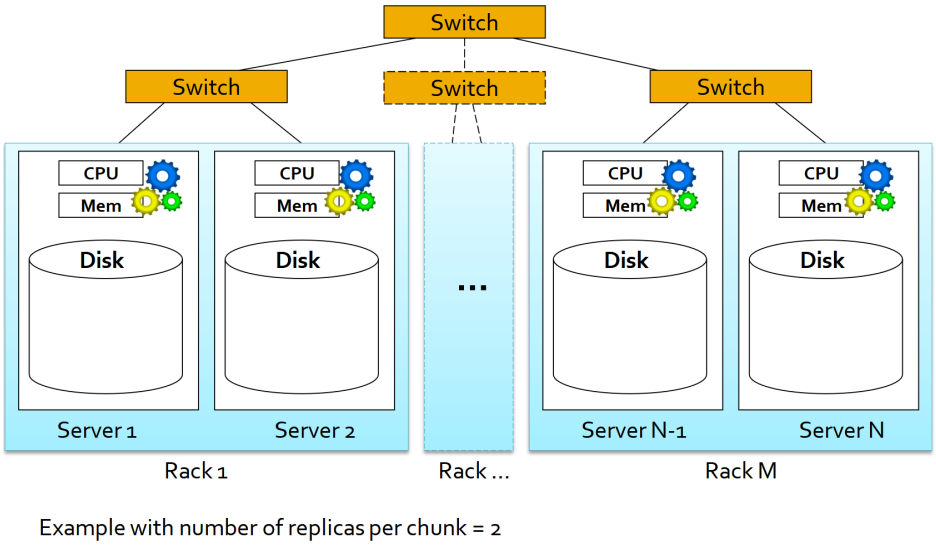


Figura 3.1: Componenti fisici principali e gerarchia di rete in un cluster Hadoop.

3.2 Il Principio della Data Locality

Nei sistemi distribuiti tradizionali (come gli ambienti HPC), l'approccio standard consiste nello spostare i dati verso i nodi di calcolo (server). Tuttavia, quando si ha a che fare con i Big Data, questo approccio non è scalabile per processare Terabyte di dati a causa dei severi limiti di larghezza di banda della rete.

Hadoop inverte questo paradigma muovendo il codice verso i dati. Il codice dell'applicazione (che pesa solitamente pochi Kilobyte) viene spedito, copiato ed eseguito direttamente sui server fisici in cui sono materialmente memorizzati i chunk (blocchi) di dati. Questo approccio fondamentale prende il nome di **Data Locality** e minimizza l'uso della rete.

3.3 Campi di Applicazione di MapReduce

Hadoop e MapReduce sono stati progettati esplicitamente per applicazioni data-intensive e processi di tipo Batch, ovvero per elaborazioni che richiedono (quasi sempre) una scansione completa di tutti i dati di input.

- **Problemi ideali per MapReduce:** MapReduce è eccellente quando si ha la necessità di iterare e scansionare enormi moli di record per estrarne informazioni. Esempi tipici includono:
  - Lettura ed elaborazione dell'intero Web (es. per il calcolo del PageRank).
  - Lettura ed elaborazione di interi grafi sociali (es. per identificare sottografi connessi e suggerire potenziali amici, noto come Link Prediction).
  - Analisi di log di grandi dimensioni (es. tracce di rete, dati provenienti da smart-meter e sensori di smart-city).
- **Limitazioni:** Hadoop/MapReduce non è la panacea per tutti i problemi legati ai Big Data. Non è adatto per:
  - Analisi di flussi di dati in streaming o elaborazioni in real-time.
  - Problemi di natura strettamente ricorsiva.

– Problemi prettamente iterativi (sebbene sia possibile implementare algoritmi come il PageRank con MapReduce, il framework non è nativamente ottimizzato per cicli di iterazione continui sugli stessi dati).

3.4 Il Paradigma di Programmazione MapReduce

Un programma Hadoop è strutturato in una sequenza rigorosa di operazioni basata su due macro-funzioni (building blocks): **Map** e **Reduce**.

Osservazione 3.1. Un singolo *job* è formato unicamente da una fase di Map seguita da una fase di Reduce. A differenza di altri sistemi (come il database NoSQL MongoDB), non è possibile concatenare fasi spurie come "Map-Reduce-Reduce" all'interno di uno stesso job. Se un task è molto complesso, deve essere suddiviso in più applicazioni MapReduce sequenziali, dove l'output del primo job diventa l'input del job successivo.

3.4.1 Struttura Dati: Coppie Chiave-Valore

La struttura dati di base in MapReduce è la coppia chiave-valore (*key-value pair*). Sia l'input che l'output di un programma MapReduce sono sempre rappresentati da liste di coppie chiave-valore. Le chiavi e i valori possono essere tipi di dato primitivi (come interi, float, stringhe) o anche strutture dati quasi arbitrarie definite appositamente dallo sviluppatore. Per poter elaborare file testuali o altre tipologie di file, esistono classi specifiche in Hadoop incaricate di leggere i dati grezzi e trasformarli in un insieme di coppie chiave-valore di input.

3.4.2 Le Fasi di Elaborazione

Ogni applicazione MapReduce è divisa internamente in tre fasi logiche:

1. **Fase di Map:** La funzione di map viene applicata a ciascun elemento del dataset di input ed emette un insieme di nuove coppie $(key, value)$ intermedie. Lo sviluppatore scrive questa funzione per trasformare logicamente il dato di input in base al task da risolvere.

2. **Fase di Shuffle and Sort:** È la fase intermedia che si occupa di raggruppare e ordinare tutto l'output della fase di map raggruppandolo per chiave.

   Osservazione 3.2. Questa fase compie sempre la stessa operazione (mettere insieme tutti i valori associati alla stessa chiave) e non deve essere implementata dallo sviluppatore, in quanto è fornita in modo automatico e altamente ottimizzato dal framework Hadoop.

3. **Fase di Reduce:** La funzione di reduce (r) può essere vista come un'operazione di aggregazione definita dal programmatore. Viene invocata esattamente una volta per ogni *chiave distinta* e si occupa di aggregare l'intera lista di valori associati a quella specifica chiave. Poiché ogni gruppo di coppie aventi la stessa chiave è totalmente indipendente dagli altri, la fase di reduce viene eseguita in isolamento ed è intrinsecamente parallelizzabile, emettendo le coppie chiave-valore che costituiranno il risultato finale.

3.5 Esempio Pratico: Il Problema del Word Count

Esempio 3.1 (Word Count)**.** Il problema del conteggio delle parole ("Word Count") rappresenta il classico esempio "Hello World" per spiegare il funzionamento delle applicazioni MapReduce. L'obiettivo è analizzare un grande file testuale in input (visto come una lista di parole) per contare il numero totale di occorrenze di ciascuna parola presente nel documento.

1. **Funzione di Map:**

La funzione di map analizza il testo di input. Per ogni parola incontrata, emette semplicemente una coppia chiave-valore dove la chiave è la parola stessa e il valore associato è 1.

$$\text{Map}(\text{input}) \rightarrow (\text{word}, 1)$$

2. **Shuffle and Sort:**

Il sistema riceve tutti i risultati della fase di map e raduna automaticamente tutti gli 1 generati per ogni singola parola.

3. **Funzione di Reduce:**

La funzione di reduce viene chiamata su una singola parola accompagnata dalla lista di tutti i valori a essa associati (che in questo caso sono tutti 1). La funzione somma semplicemente tutti i valori dell'array e restituisce una singola coppia chiave-valore contenente la parola e il suo numero globale di occorrenze.

$$\text{Reduce}(\text{word}, [1, 1, \ldots, 1]) \rightarrow (\text{word}, \text{numero totale di occorrenze})$$

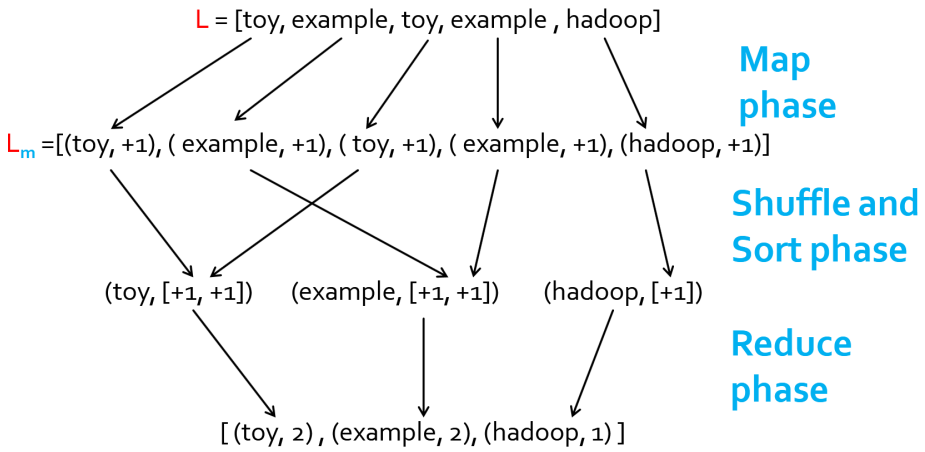


Figura 3.2: Esempio di applicazione MapReduce per il problema del Word Count.

Capitolo 4

Sviluppo di Applicazioni MapReduce in Hadoop

Per implementare ed eseguire un'applicazione basata sul paradigma MapReduce all'interno dell'ecosistema Hadoop, è necessario utilizzare il linguaggio di programmazione Java e interfacciarsi con le API (Application Programming Interface) fornite dal framework. Lo sviluppo di un programma MapReduce richiede la definizione di tre componenti fondamentali, ciascuna implementata attraverso una specifica classe Java: la classe Driver, la classe Mapper e la classe Reducer.

4.1 Il Ruolo del Framework e dello Sviluppatore

Nello sviluppo di applicazioni MapReduce, vi è una netta separazione delle responsabilità tra lo sviluppatore e il framework Hadoop. Il compito principale dei progettisti e degli sviluppatori è concentrarsi esclusivamente sulla definizione della logica applicativa, ovvero sull'implementazione delle funzioni di Map e di Reduce (spesso indicate come m e r).

Lo sviluppatore **non deve gestire in alcun modo l'esecuzione distribuita**. È il framework Hadoop a coordinare in totale autonomia l'intera esecuzione del programma MapReduce. Nello specifico, Hadoop si occupa di:

- Gestire l'esecuzione parallela delle fasi di map e di reduce.
- Eseguire automaticamente la fase intermedia di *shuffle and sort*.
- Gestire lo scheduling dei vari subtask sui nodi del cluster.
- Occuparsi della sincronizzazione tra i processi.

4.2 Architettura di un Programma MapReduce

Il framework impone una precisa distinzione terminologica e concettuale tra le classi (che definiscono la logica del programma) e le loro istanze (gli oggetti effettivamente eseguiti in parallelo sul cluster).

4.2.1 Definizione delle Classi

- **Driver class:** È la classe principale (spesso contenente il metodo `main`) in cui si forniscono le informazioni relative al flusso di lavoro (workflow) dell'applicazione. Al suo interno viene configurato il *Job* MapReduce, specificando quali classi utilizzare

per le fasi di map e reduce, i percorsi dei dati di input e output e altre proprietà di esecuzione.

- **Mapper class:** È la classe dedicata all'implementazione della *map phase*. Lo sviluppatore deve scrivere la logica di elaborazione all'interno del metodo `map`.
- **Reducer class:** È la classe dedicata all'implementazione della *reduce phase*. La logica di aggregazione viene definita sovrascrivendo il metodo `reduce`.

4.2.2 Istanze e Terminologia

Quando il codice viene mandato in esecuzione, il sistema crea diverse istanze di queste classi. Nel gergo di Hadoop:

- Il termine **Driver** si riferisce all'oggetto istanziato dalla Driver class.
- Il termine **Mapper** (senza ulteriori specificazioni) indica un'istanza della Mapper class. Ogni Mapper è incaricato di analizzare una specifica porzione dei dati di input.
- Il termine **Reducer** indica un'istanza della Reducer class. Ciascun Reducer gestisce la fase di aggregazione per uno specifico sottoinsieme di chiavi generate dalla fase di map.

Infine, un **Job** rappresenta l'esecuzione vera e propria (la *run*) dell'applicazione MapReduce sul cluster Hadoop.

4.2.3 Il Driver come Entry Point

La classe Driver contiene il metodo `main()`, che accetta gli argomenti passati da riga di comando. Esso rappresenta l'*entry point* (punto di ingresso) dell'applicazione. All'interno del Driver, il codice configura il Job e, infine, lo sottomette al cluster Hadoop per l'esecuzione effettiva.

4.3 Gestione dell'Output Intermedio e Finale

Durante l'esecuzione delle fasi di MapReduce, vi è una profonda differenza nel modo in cui vengono memorizzati i risultati generati dai Mapper e quelli generati dai Reducer.

4.3.1 Dati Intermedi (Transient Data)

Il metodo map del Mapper viene invocato su ogni singola coppia chiave-valore di input e genera un insieme di coppie chiave-valore intermedie. È fondamentale notare che questi risultati intermedi **sono transitori e non vengono salvati sull'HDFS**. Essi vengono memorizzati esclusivamente sul *file system locale* del nodo (computing server) che li ha prodotti o elaborati. Successivamente, questi dati intermedi vengono aggregati tramite la procedura di *shuffle and sort*, che genera per ogni singola chiave distinta una coppia formata dalla chiave stessa e dalla relativa lista di valori: `(key, [list of values])`.

4.3.2 Dati Finali

Il metodo reduce viene applicato proprio su queste coppie `(key, [list of values])` aggregate. Al contrario dei dati intermedi, le coppie chiave-valore emesse in output dai Reducer rappresentano il risultato finale dell'applicazione MapReduce e **vengono memorizzate in modo permanente all'interno dell'HDFS**.

4.4 Lettura dei Dati: La classe InputFormat

Il punto di partenza di un programma MapReduce è l'acquisizione dei dati in ingresso. L'input dell'applicazione è solitamente costituito da un file (o da un'intera cartella) residente all'interno dell'HDFS (Hadoop Distributed File System). Tuttavia, il metodo map del Mapper non legge file testuali grezzi, ma richiede in ingresso un flusso strutturato come un insieme di coppie chiave-valore `(key, value)`.

Per colmare questa distanza interviene la classe astratta `org.apache.hadoop.mapreduce.InputFormat`. Le classi che estendono `InputFormat` hanno un duplice compito:

1. Descrivere le specifiche del formato di input per l'applicazione MapReduce e processare i file di ingresso.

2. Leggere i dati di input e **"trasformare logicamente"** il file HDFS in un insieme di coppie chiave-valore che possano essere fornite direttamente alle istanze dei Mapper.

4.5 Parallelismo ed Esecuzione: Task e Input Split

Per comprendere come Hadoop parallelizza l'elaborazione, è fondamentale introdurre il concetto di **Input Split**. Quando si elabora un dataset di grandi dimensioni, questo non viene letto per intero da un singolo processo. Il dataset (memorizzato nell'HDFS - Hadoop Distributed File System) è fisicamente suddiviso in blocchi (Data Blocks).

L'Input Split è una partizione logica dei dati di input che deve essere elaborata da una singola istanza Mapper. Generalmente, vi è una corrispondenza uno-a-uno tra un Input Split e un blocco fisico memorizzato nel file system distribuito.

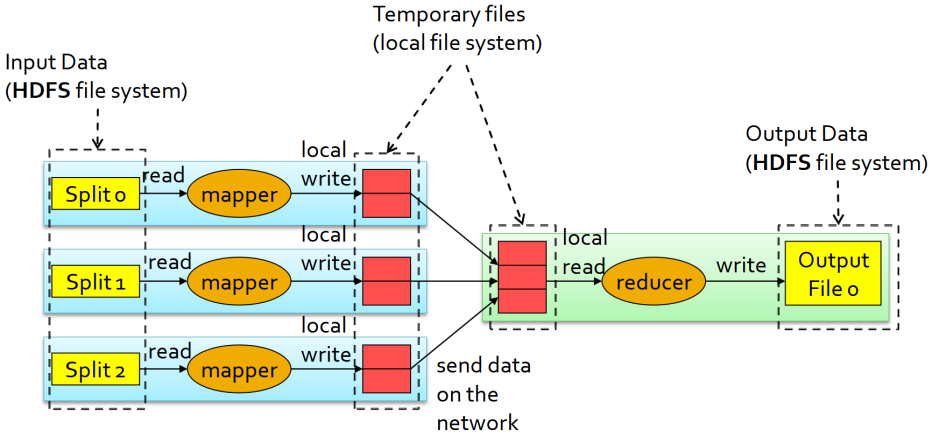


Figura 4.1: Architettura di un'applicazione MapReduce in Hadoop.

4.5.1 Grado di Parallelismo della fase di Map

Il numero di istanze della classe Mapper (ovvero il numero di task di map) che il sistema crea in parallelo è esattamente **uguale al numero di Input Split**. Se, ad esempio, il file di input è suddiviso logicamente in 10 blocchi, il sistema istanzierà 10 Mapper. Ognuna di queste istanze prenderà in carico uno split.

Questo rappresenta il **massimo parallelismo ottenibile senza dover inviare dati attraverso la rete** (network shuffle). L'elaborazione in questa fase è puramente locale al nodo che detiene il blocco di dati. Ciascun task analizza un sottoinsieme delle coppie chiave-valore di input e genera un sottoinsieme delle coppie chiave-valore intermedie.

4.6 Tipi di Dato in MapReduce

Le coppie chiave-valore scambiate tra Mapper e Reducer non utilizzano i tipi di dato base standard di Java (come `String`, `Integer`, ecc.). Hadoop definisce dei propri tipi di dato di base, ottimizzati per la **serializzazione di rete** (network serialization), poiché i dati intermedi devono viaggiare attraverso il cluster.

Tra i tipi principali (che implementano l'interfaccia `Writable`) troviamo:

- `org.apache.hadoop.io.Text`: l'equivalente della classe `String` in Java.
- `org.apache.hadoop.io.IntWritable`: l'equivalente della classe `Integer`.
- `org.apache.hadoop.io.LongWritable`: l'equivalente della classe `Long`.
- `org.apache.hadoop.io.FloatWritable`: l'equivalente della classe `Float`.

4.6.1 Tipi di Dato Personalizzati (Personalized Data Types)

A volte i tipi di dato base non sono sufficienti a modellare l'informazione. Ad esempio, si potrebbe voler trasmettere come valore non un singolo intero, ma una struttura complessa composta da una coppia di numeri, coordinate o altri dati aggregati. In questi casi, il framework permette allo sviluppatore di definire **tipi di dato personalizzati**, creando classi apposite che implementano i meccanismi di serializzazione richiesti da Hadoop.

4.6.2 Tipi di Dato Personalizzati (Personalized Data Types)

Oltre ai tipi predefiniti da Hadoop (come `Text` o `IntWritable`, ottimizzati per la serializzazione di rete), gli sviluppatori hanno la possibilità di definire nuovi **tipi di dato personalizzati** per modellare informazioni e strutture complesse.

Ciò si ottiene creando classi che implementano le interfacce `org.apache.hadoop.io.Writable` e/o `org.apache.hadoop.io.WritableComparable`. A questo proposito, il professore evidenzia un requisito tecnico vitale e categorico: se il tipo di dato personalizzato deve essere utilizzato **come chiave** (key part), non basta estendere la semplice interfaccia `Writable`, ma **è obbligatorio implementare anche i metodi associati a `WritableComparable`**. Se si omette l'implementazione di questi due metodi di comparazione, il sistema non è in grado di gestire i dati e genererà un errore a runtime durante l'esecuzione. Il framework vi avviserà indicando l'impossibilità di calcolare l'hash della chiave ("*I don't know how to hash the key*") e di memorizzare correttamente le coppie chiave-valore ("*I don't know how to store the pairs*").

4.7 Implementazione del Reducer

La classe che implementa il Reducer deve estendere la classe generica `org.apache.hadoop.mapreduce.Reducer`. Questa classe richiede la definizione di quattro parametri di tipo (generics) che corrispondono alle firme dei dati in ingresso e in uscita:

1. `Input key type` (Tipo della chiave intermedia proveniente dal Mapper).
2. `Input value type` (Tipo del valore intermedio).
3. `Output key type` (Tipo della chiave finale emessa).
4. `Output value type` (Tipo del valore finale emesso).

Lo sviluppatore deve sovrascrivere il metodo `reduce(...)`. Questo metodo viene chiamato **automaticamente dal framework per ogni singola chiave** e per la relativa lista di valori generata dall'aggregazione degli output di tutti i Mapper. Il metodo processa i dati usando codice Java standard e, al termine dell'aggregazione, emette le coppie chiave-valore finali utilizzando l'oggetto `Context` tramite il metodo `context.write(key, value)`.

4.7.1 Esempio: Word Count Reducer

Di seguito è illustrata l'implementazione del metodo `reduce` per il classico problema del Word Count:

Listing 4.1: Implementazione del metodo reduce per il Word Count

```
/* Implementazione del metodo reduce */
protected void reduce(
    Text key, // Input key type (la parola)
    Iterable<IntWritable> values, // Input value type (lista di occorrenze)
    Context context) throws IOException, InterruptedException {

    int occurrences = 0;

    // Itera sull'insieme dei valori e li somma
    for (IntWritable value : values) {
        occurrences = occurrences + value.get();
    }

    // Emette il numero totale di occorrenze per la parola corrente
    context.write(key, new IntWritable(occurrences));
} // Fine metodo reduce
```

Come si evince dal codice, per ogni parola (la `key`), il framework fornisce un iteratore ai valori (gli "1" emessi dai mapper). Il reducer li somma e produce il totale finale.

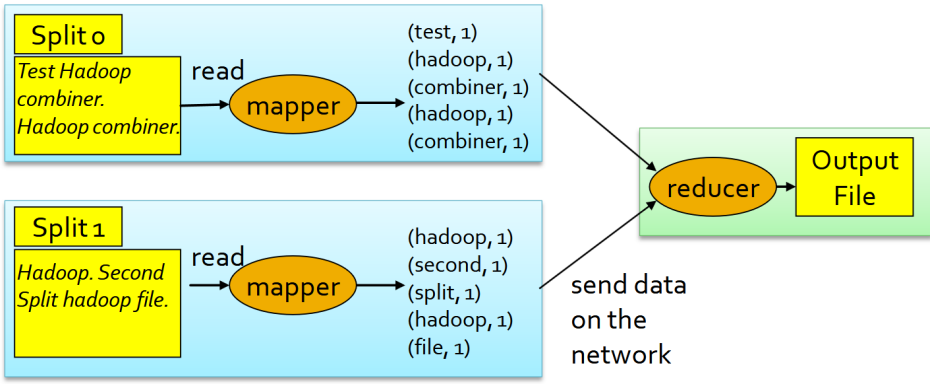


Figura 4.2: Esecuzione della fase di reduce.

4.8 L'Uso esclusivo del Map: Map-Only Job

Esistono applicazioni pratiche in cui l'aggregazione dei dati (fase di reduce) non è necessaria. L'obiettivo potrebbe essere semplicemente quello di filtrare dati, pulire un dataset o trasformare i record riga per riga.

Il professore illustra un caso d'uso: supponiamo di avere un file di log proveniente da sensori, con il formato `<sensor_ID> <date>`. Tuttavia, il dataset è sporco e alcune righe

presentano formati errati (es. solo l'ID del sensore senza la data, oppure un'anomalia come "Paolo 50"). Se la riga è corretta o utile ai fini della nostra analisi (es. fa parte del risultato che vogliamo conservare), dobbiamo emetterla così com'è. Se la riga è malformata o inutile, la ignoriamo non restituendo nulla.

In questo scenario, non stiamo aggregando valori considerando più righe contemporaneamente. Stiamo solo decidendo, riga per riga, se un dato è utile o meno. Per questo scopo è sufficiente un **Map-Only Job**. In un Map-Only Job viene eseguita esclusivamente la fase di map; il framework salta del tutto le fasi di shuffle, sort e reduce. L'output generato dalle istanze dei Mapper viene scritto direttamente nella cartella di destinazione (output folder). Questo approccio ottimizza notevolmente l'esecuzione, poiché **evita di inviare enormi moli di dati sulla rete** (network traffic) verso Reducer che, di fatto, non dovrebbero compiere alcuna aggregazione.

4.9 Ottimizzazione del Traffico di Rete: Il Combiner

Durante l'esecuzione standard di un programma MapReduce, tutte le coppie emesse dai Mapper vengono inviate via rete ai Reducer. Se consideriamo il problema del Word Count su un testo enorme, l'invio di innumerevoli coppie del tipo `("parola", 1)` genererà una pesante congestione della rete.

Come si può ridurre la mole di dati scambiata sulla rete? La soluzione offerta da Hadoop è il **Combiner**. Il Combiner agisce come un "Mini-Reducer" locale. Viene eseguito sullo stesso nodo del Mapper e aggrega preliminarmente l'output del Mapper *prima* che questo venga trasmesso sulla rete. Nel Word Count, invece di mandare cento coppie `("parola", 1)`, il Combiner le sommerà localmente inviando alla rete una singola coppia `("parola", 100)`. L'informazione semantica resta la medesima, ma il traffico di rete viene drasticamente ridotto, incrementando l'efficienza.

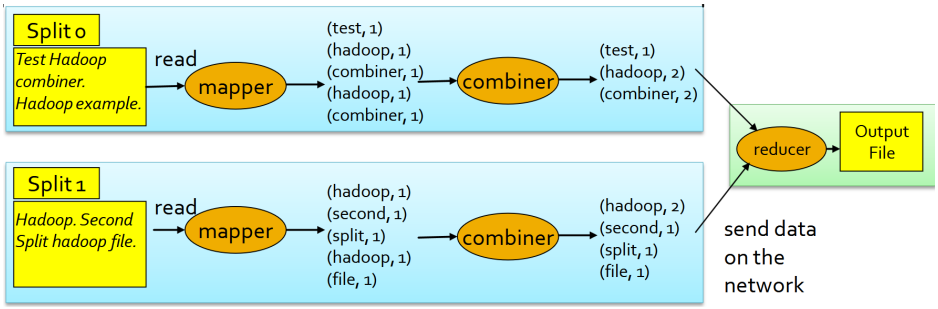


Figura 4.3: Esecuzione del Combiner per ridurre il traffico di rete.

4.9.1 Implementazione del Combiner

Spesso, la logica di aggregazione del Combiner è identica a quella del Reducer. Nel caso del Word Count, non è necessario scrivere due classi diverse. Si può istruire il Driver a usare la classe del Reducer anche come Combiner tramite il comando:

```
job.setCombinerClass(WordCountReducer.class);
```

Nella stragrande maggioranza dei casi (il 99% delle applicazioni Hadoop), la medesima classe implementa in modo sicuro ed efficiente sia il Combiner che il Reducer (a patto che l'operazione matematica sottostante sia commutativa e associativa).

4.10 Ottimizzazione Avanzata: L'In-Mapper Combiner

Un'ulteriore evoluzione, volta a ottimizzare in maniera ancor più aggressiva l'esecuzione, è rappresentata dall'approccio **In-Mapper Combiner**. L'idea alla base è quella di eseguire l'aggregazione direttamente all'interno della logica del Mapper, sfruttando variabili di stato interne (in-mapper variables).

Per implementare questa logica si sfruttano due metodi accessori della classe Mapper (e Reducer):

- `setup(...)`: Metodo invocato dal framework **una sola volta** per ciascun task, prima che inizino le ripetute chiamate al metodo `map`/`reduce`.
- `cleanup(...)`: Metodo invocato dal framework **una sola volta** per ciascun task, dopo che tutti i record di input sono stati processati.

4.10.1 Workflow dell'In-Mapper Combiner

Il flusso di lavoro si sviluppa come segue:

1. **Inizializzazione**: Nel metodo `setup` del Mapper, si inizializzano delle strutture dati locali (es. una mappa in memoria o delle *in-mapper variables*) per mantenere le statistiche o le aggregazioni parziali.
2. **Aggiornamento**: Nel metodo `map`, invece di emettere le coppie con `context.write`, si aggiornano semplicemente le strutture dati interne. In questa fase, **nessuna coppia viene emessa**.
3. **Emissione finale**: Una volta elaborati tutti i record di competenza del Mapper, il framework chiama il metodo `cleanup`. Al suo interno, si leggono i risultati aggregati presenti nelle variabili locali e li si emette sotto forma di coppie chiave-valore.

4.10.2 Considerazioni Critiche sulle Prestazioni e Memoria Principale

Sebbene l'In-Mapper Combiner consenta di migliorare sensibilmente le prestazioni globali dell'applicazione (evitando l'istanziazione continua di oggetti e chiamate ripetute all'API di emissione), introduce una limitazione fondamentale sollevata dal professore: **il consumo della memoria principale (RAM)**.

A ogni task Mapper è allocata una quantità limitata e predefinita di memoria principale. Poiché le variabili interne di aggregazione crescono a mano a mano che vengono scoperte nuove chiavi, **le strutture dati in-mapper devono rimanere di dimensioni ridotte**. Se lo spazio occupato dalle variabili locali supera il limite massimo di RAM assegnato al task Mapper, si causerà un errore di tipo Out-Of-Memory, provocando il fallimento del task.

4.11 Gestione del Contesto e Statistiche

4.11.1 Condivisione di Parametri globali

In alcuni casi, è necessario condividere piccoli parametri o proprietà di configurazione che rimangono costanti durante tutta l'esecuzione del programma. Questo si ottiene definendo coppie `(property-name, property-value)`.

- **Nel Driver:** Lo sviluppatore recupera l'oggetto di configurazione e imposta le proprietà:

```
Configuration conf = this.getConf();
conf.set("property-name", "value");
```

- **Nel Mapper/Reducer:** L'oggetto `Context` permette di accedere in sola lettura a queste proprietà:

```
String val = context.getConfiguration().get("property-name");
```

È fondamentale notare che i Mapper e i Reducer **non possono modificare** i valori di queste proprietà durante l'esecuzione.

4.11.2 Contatori (Counters)

I Counters sono numeri interi gestiti direttamente dal framework, utili per raccogliere ed esporre statistiche diagnostiche sull'andamento dei Job, Mappers e Reducers. Hadoop fornisce una serie di **built-in counters** (contatori predefiniti) che tracciano metriche base (es. record in lettura, byte scritti).

Inoltre, lo sviluppatore può definire **contatori personalizzati** per tenere traccia di logiche di business specifiche. Riprendendo l'esempio dei file di log dei sensori malformati discusso dal professore, se incontriamo una riga incompleta (senza la data), potremmo non solo scartarla, ma anche incrementare un contatore personalizzato (es. "Linee Malformate"). A fine elaborazione, potremo leggere questo contatore per calcolare semplici statistiche sulla qualità dei dati di input.